

Algèbre Linéaire en Matlab

15 novembre 2023

1 Résolution de systèmes d'équations linéaires

1.1 Méthode de Gauss-Jordan

La méthode de Gauss-Jordan est une technique de résolution de systèmes d'équations linéaires :

$$\begin{pmatrix} a_{1,1} & a_{1,2} & \cdots & a_{1,n} \\ a_{2,1} & a_{2,2} & \cdots & a_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n,1} & a_{n,2} & \cdots & a_{n,n} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix} \quad (1)$$

Elle consiste à transformer la matrice associée $A = (a_{ij})_{1 \leq i \leq n, 1 \leq j \leq n}$ en une matrice unité en effectuant des combinaisons linéaires de lignes. Cette variante de la méthode de Gauss permet de calculer l'inverse d'une matrice.

1.2 Matrice augmentée

Soient A et B deux matrices ayant le même nombre de lignes, on appelle matrice augmentée la matrice $(A|B)$ formée des deux blocs A et B mis côte à côte.

Cette notion est très pratique lorsqu'on veut résoudre le système d'équations avec différents seconds membres. La matrice B est alors construite en accolant à droite de A , les vecteurs colonnes formant les seconds membres. Elle se réduit naturellement à un vecteur dans le cas d'un seul second membre.

1.3 Algorithme de base sans changement de pivot (niveau *)

On augmente la matrice A avec le(s) vecteur(s) colonne(s) du second membre. On vérifie que $\forall i, a_{ii} \neq 0$ sinon effectue des échanges de lignes.

L'algorithme est itératif et s'effectue en n étapes. A l'étape $k \in [1, n]$, on combine toutes les lignes sauf la ligne k pour faire apparaître des 0 sur toute la colonne k sauf au niveau du pivot a_{kk} .

```
pour k=1 à n
    diviser la ligne k de la matrice A par akk
    pour i=1 à n sauf k
        retrancher à la ligne i, la nouvelle ligne k multipliée par aik
    fin pour i
fin pour k
```

Après résolution, la matrice A apparait comme la concaténation de la matrice identité, accolée aux solutions x du système.

1.4 Exemple pratique

Soit à résoudre :

$$\begin{cases} 2x + y - 4z = 8 \\ 3x + 3y - 5z = 14 \\ 4x + 5y - 2z = 16 \end{cases}$$

La matrice augmentée $A^{(0)}$ s'écrit :

$$A^{(0)} = \left(\begin{array}{ccc|c} 2 & 1 & -4 & 8 \\ 3 & 3 & -5 & 14 \\ 4 & 5 & -2 & 16 \end{array} \right) \quad (2)$$

Divisons la première ligne par le pivot $a_{1,1} : l_1 \leftarrow \frac{1}{2} \times l_1$

$$A^{(1)} = \left(\begin{array}{ccc|c} 1 & 1/2 & -2 & 4 \\ 0 & 3/2 & 1 & 2 \\ 0 & 3 & 6 & 0 \end{array} \right) \begin{array}{l} l_2 \leftarrow l_2 - 3 \times l_1 \\ l_3 \leftarrow l_3 - 4 \times l_1 \end{array} \quad (3)$$

Divisons la deuxième ligne par le pivot $a_{2,2} : l_2 \leftarrow \frac{2}{3} \times l_2$

$$A^{(2)} = \left(\begin{array}{ccc|c} 1 & 0 & -7/3 & 10/3 \\ 0 & 1 & 2/3 & 4/3 \\ 0 & 0 & 4 & -4 \end{array} \right) \begin{array}{l} l_1 \leftarrow l_1 - \frac{1}{2} \times l_2 \\ l_3 \leftarrow l_3 - 3 \times l_2 \end{array} \quad (4)$$

Divisons la troisième ligne par le pivot $a_{3,3} : l_3 \leftarrow \frac{1}{4} \times l_3$

$$A^{(3)} = \left(\begin{array}{ccc|c} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & -1 \end{array} \right) \begin{array}{l} l_1 \leftarrow l_1 + \frac{7}{3} \times l_3 \\ l_2 \leftarrow l_2 - \frac{2}{3} \times l_3 \end{array} \quad (5)$$

La solution se trouve dans la quatrième colonne. Vérifier que $x = (1, 2, -1)^t$ est bien solution.

1.5 Inverse d'une matrice (niveau **)

Le calcul direct de l'inverse d'une matrice A mettant en oeuvre la formule $A^{-1} = \frac{1}{\det(A)} \text{co}(A)$ est peu utilisé en pratique. On préfère résoudre n systèmes linéaires en même temps en plaçant dans le second membre la matrice identité I_n . On applique la méthode de Gauss-Jordan à la matrice augmentée $(A|I_n)$.

- Montrer que l'inverse de la matrice

$$A = \begin{pmatrix} 2 & 1 & -4 \\ 3 & 3 & -5 \\ 4 & 5 & -2 \end{pmatrix} \text{ est } A^{-1} = \frac{1}{12} \begin{pmatrix} 19 & -18 & 7 \\ -14 & 12 & -2 \\ 3 & -6 & 3 \end{pmatrix}. \text{ Vérifier que } AA^{-1} = I.$$

1.6 Implémentation de Gauss-Jordan en Matlab (niveau *)

1. Proposez une implémentation d'une fonction `gauss-jordan` en Matlab de l'algorithme précédent, permettant de résoudre un système d'équations linéaires.
2. Traitez quelques cas tests, vérifiez vos résultats.
3. On étudie ici la stabilité numérique de l'algorithme en prenant :

$$A = \begin{pmatrix} 1 & 1/4 & 0 & 0 \\ 1 & 5/4 & 12 & 0 \\ 1 & 1/3 & 1 & 1 \\ 1 & 5/4 & 13 & 1 \end{pmatrix} \text{ et } B = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix}$$

Calculer le résidu $B - Ax$ dans ce cas, commenter.

1.7 Algorithme de Gauss-Jordan avec changement de pivot (niveau **)

On augmente la matrice A avec le(s) vecteur(s) colonne(s) du second membre. On propose une nouvelle version de l'algorithme de Gauss-Jordan avec maximisation du pivot. L'algorithme est itératif et s'effectue en n étapes. A l'étape $k \in [1, n]$, on combine toutes les lignes sauf la ligne k pour faire apparaître des 0 sur toute la colonne k sauf au niveau du pivot a_{kk} .

```

pour k=1 à n
    identifier l'indice de ligne  $i \in [k, n]$  correspondant au  $\max\{|a_{i,k}|\}$ 
    échanger les lignes  $i$  et  $k$  de la matrice  $A$ 
    diviser ligne  $k$  de la nouvelle matrice  $A$  par  $a_{kk}$ 
    pour i=1 à n sauf k
        retrancher à la ligne  $i$ , la nouvelle ligne  $k$  multipliée par  $a_{ik}$ 
    fin pour i
fin pour k
    
```

1. traiter quelques cas tests.
2. refaites l'étude de la stabilité numérique.

1.8 Résolution de systèmes mal conditionnés (niveau **)

On examine ici l'influence d'une perturbation δb sur les termes du second membre b du système d'équations linéaires. Il en résulte une variation δx sur la solution. On montre par contre que δx n'est pas forcément une perturbation de x et dépend du conditionnement de A . On a l'inégalité suivante :

$$\frac{\|\delta x\|}{\|x\|} < \text{cond}(A) \frac{\|\delta b\|}{\|b\|} \quad (6)$$

Le conditionnement est donné dans le cas d'une matrice symétrique par le rapport des valeurs propres extrêmes en norme de A : $\kappa(A) = \frac{|\lambda_{\max}(A)|}{|\lambda_{\min}(A)|}$.

1. Résoudre le système d'équations en prenant : $A = \begin{pmatrix} 10 & 7 & 8 & 7 \\ 7 & 5 & 6 & 5 \\ 8 & 6 & 10 & 9 \\ 7 & 5 & 9 & 10 \end{pmatrix}$ et $B = \begin{pmatrix} 32 \\ 23 \\ 33 \\ 31 \end{pmatrix}$
2. Faites une fluctuation de $\pm 1\%$ sur le second membre et en déduire l'erreur sur la solution.
3. Calculer le conditionnement de A à partir de son spectre. On utilisera la fonction Matlab `eig` pour calculer les valeurs propres.
4. Pour vérifier l'inégalité (6), on calculera les normes avec la fonction `norm` en choisissant une norme infinie, par exemple.

2 Décomposition LU d'une matrice sans pivot (niveau **)

Le but est ici de décomposer une matrice sous la forme : $A = L U$ où L une matrice triangulaire inférieure ne contenant que des 1 sur sa diagonale et U une matrice triangulaire supérieure.

A titre d'exemple, factorisons la matrice $A = \begin{pmatrix} 10 & -7 & 0 \\ -3 & 2 & 6 \\ 5 & -1 & 5 \end{pmatrix}$

La première étape d'élimination consiste à faire les combinaisons linéaires suivantes :

$$\begin{aligned} l_1 &\leftarrow l_1 \\ l_2 &\leftarrow l_2 + 3/10 \times l_1 \\ l_3 &\leftarrow l_3 - 1/2 \times l_1 \end{aligned} \quad (7)$$

$$A^{(1)} = L_1^{-1} A^{(0)} = \begin{pmatrix} 1 & 0 & 0 \\ 3/10 & 1 & 0 \\ -1/2 & 0 & 1 \end{pmatrix} \begin{pmatrix} 10 & -7 & 0 \\ -3 & 2 & 6 \\ 5 & -1 & 5 \end{pmatrix} = \begin{pmatrix} 10 & -7 & 0 \\ 0 & -1/10 & 6 \\ 0 & 5/2 & 5 \end{pmatrix} \quad (8)$$

La seconde étape d'élimination consiste à faire l'opération suivante :

$$l_3 \leftarrow l_3 + 25 \times l_2 \quad (9)$$

$$U = A^{(2)} = L_2^{-1} A^{(1)} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 25 & 1 \end{pmatrix} \begin{pmatrix} 10 & -7 & 0 \\ 0 & -1/10 & 6 \\ 0 & 5/2 & 5 \end{pmatrix} = \begin{pmatrix} 10 & -7 & 0 \\ 0 & -1/10 & 6 \\ 0 & 0 & 155 \end{pmatrix} \quad (10)$$

Le calcul des matrices L_k s'effectue facilement grâce à la propriété suivante :

$$L_k = -L_k^{-1} + 2I_n \quad (11)$$

La matrice triangulaire inférieure L est finalement reconstruite en appliquant la relation suivante :

$$L = \Pi_{k=1}^{n-1} L_k \quad (12)$$

On fournit l'algorithme suivant :

```

1: function LU(A)                                9:      A ← MULT(invLk, A)
2:   L ← In                                       10:     Lk ← -invLk + 2In
3:   for k ∈ [1, n - 1] do                        11:     L ← MULT(L, Lk)
4:     invLk ← In                                12:   end for
5:     for i ∈ [k + 1, n] do                        13:
6:       invLk(i, k) ← -ai,k/ak,k                14:   U ← A
7:     end for                                       15:   return L, U
8:                                                     16: end function
    
```

1. Proposez une implémentation en Matlab.
2. Testez-la en utilisant les matrices fournies précédemment.
3. Que se passe-t-il quand le pivot est trop petit voire nul ?

3 Décomposition LU d'une matrice avec pivot (niveau ***)

Le but est ici de décomposer une matrice sous la forme : $A = P L U$ où P est une matrice de permutation de lignes de la matrice unité, L une matrice triangulaire inférieure ne contenant que des 1 sur sa diagonale et U une matrice triangulaire supérieure. La matrice de permutation est une matrice orthogonale et vérifie la propriété $P^{-1} = P^t$.

A titre d'exemple, factorisons la matrice $A = \begin{pmatrix} 10 & -7 & 0 \\ -3 & 2 & 6 \\ 5 & -1 & 5 \end{pmatrix}$

Déterminons le pivot qui correspond à l'amplitude maximale des termes de la première sous-colonne. La première étape de permutation est triviale parce que la valeur 10 est déjà la plus grande valeur. la première matrice de permutation est l'identité.

$$P_1 A^{(0)} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 10 & -7 & 0 \\ -3 & 2 & 6 \\ 5 & -1 & 5 \end{pmatrix} = \begin{pmatrix} 10 & -7 & 0 \\ -3 & 2 & 6 \\ 5 & -1 & 5 \end{pmatrix} \quad (13)$$

La première étape d'élimination consiste à faire les combinaisons linéaires suivantes :

$$\begin{aligned} l_1 &\leftarrow l_1 \\ l_2 &\leftarrow l_2 + 3/10 \times l_1 \\ l_3 &\leftarrow l_3 - 1/2 \times l_1 \end{aligned} \quad (14)$$

$$A^{(1)} = L_1^{-1} P_1 A^{(0)} = \begin{pmatrix} 1 & 0 & 0 \\ 3/10 & 1 & 0 \\ -1/2 & 0 & 1 \end{pmatrix} \begin{pmatrix} 10 & -7 & 0 \\ -3 & 2 & 6 \\ 5 & -1 & 5 \end{pmatrix} = \begin{pmatrix} 10 & -7 & 0 \\ 0 & -1/10 & 6 \\ 0 & 5/2 & 5 \end{pmatrix} \quad (15)$$

La seconde étape de permutation revient à échanger les lignes 2 et 3 :

$$P_2 A^{(1)} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} 10 & -7 & 0 \\ 0 & -1/10 & 6 \\ 0 & 5/2 & 5 \end{pmatrix} = \begin{pmatrix} 10 & -7 & 0 \\ 0 & 5/2 & 5 \\ 0 & -1/10 & 6 \end{pmatrix} \quad (16)$$

La seconde étape d'élimination consiste à faire l'opération suivante :

$$l_3 \leftarrow l_3 + \frac{1}{25} \times l_2 \quad (17)$$

On obtient :

$$U = A^{(2)} = L_2^{-1} P_2 A^{(1)} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1/25 & 1 \end{pmatrix} \begin{pmatrix} 10 & -7 & 0 \\ 0 & 5/2 & 5 \\ 0 & -1/10 & 6 \end{pmatrix} = \begin{pmatrix} 10 & -7 & 0 \\ 0 & 5/2 & 5 \\ 0 & 0 & 31/5 \end{pmatrix} \quad (18)$$

D'où :

$$U = L_2^{-1} P_2 L_1^{-1} P_1 A \equiv (PL)^{-1} A \quad (19)$$

On en déduit :

$$PL = \Pi_{k=1}^{n-1} (P_k^t L_k) = \Pi_{k=1}^{n-1} (P_k L_k) \quad (20)$$

en remarquant que chaque matrice P_k est symétrique. On construit finalement la matrice globale de permutation $P = \Pi_{k=1}^{n-1} P_k$ qui n'est pas en général symétrique.

Le calcul des matrices L_k s'effectue facilement grâce à la propriété suivante :

$$L_k = -L_k^{-1} + 2I_n \quad (21)$$

La matrice triangulaire inférieure L est finalement reconstruite sachant que $P^{-1} = P^t$:

$$L = P^t \Pi_{k=1}^{n-1} (P_k^t L_k) = P^t \Pi_{k=1}^{n-1} (P_k L_k) \quad (22)$$

Dans l'algorithme fourni ci-après, on définit une fonction `find_pivot(A, k)` qui retourne pour une colonne k donné, l'indice de ligne $i \in [k, n]$ correspondant au $\max\{|a_{i,k}|\}$.

```

1: function PLU(A)                                14:
2:   PL ← In                                       15:   A ← MULT(invLk, A)
3:   P ← In                                       16:   Lk ← -invLk + 2In
4:   for k ∈ [1, n - 1] do                        17:   PLk ← MULT(Pk, Lk)
5:     Pk ← In                                   18:   PL ← MULT(PL, PLk)
6:     i ← FIND_PIVOT(A, k)                       19:   P ← MULT(P, Pk)
7:     Pk ← SWAP_LINES(Pk, i, k)                20:   end for
8:     A ← SWAP_LINES(A, i, k)                    21:
9:   invLk ← In                                   22:   U ← A
10:  for i ∈ [k + 1, n] do                         23:   invP ← TRANSPOSE(P)
11:    invLk(i, k) ← -ai,k/ak,k                 24:   L ← MULT(invP, PL)
12:  end for                                       25:   return P, L, U
13: end function                                26: end function
    
```

1. Proposez une implémentation en Matlab.
2. Testez-la en utilisant les matrices fournies précédemment.
3. Proposez un exemple qui mettrait en défaut la simple factorisation LU.